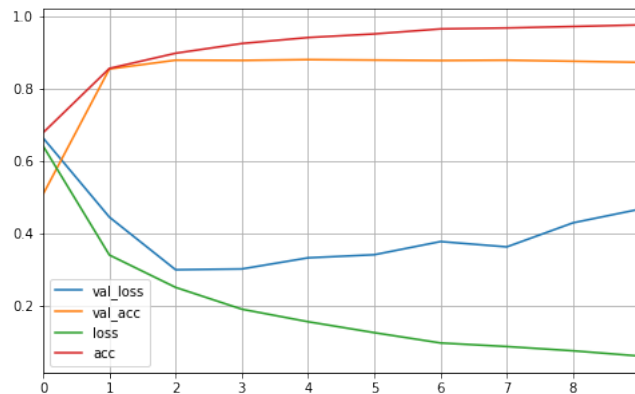


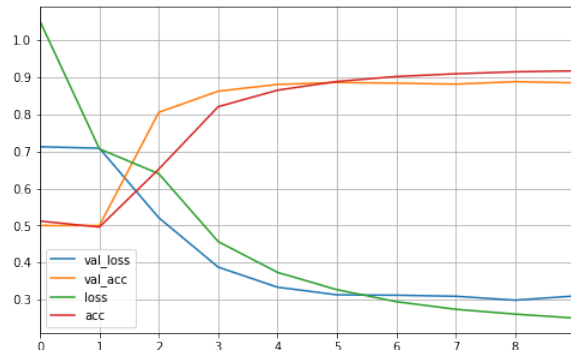
Lab 6

3. A baseline model of 1D CNN is constructed and trained on IMDB dataset for 10 epochs displayed as below.

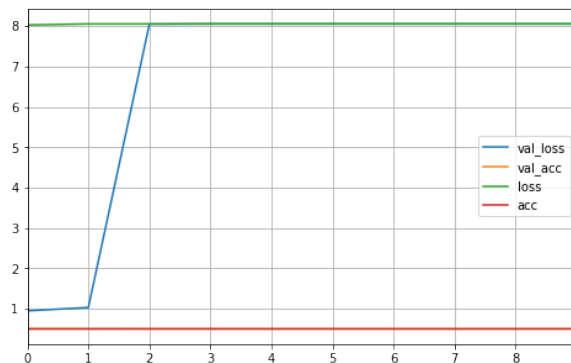


It could be observed that the model begins to overfit after 2-3 epochs with validation loss starting to rise and validation accuracy unchanged. To reduce overfitting, some approaches could be applied such as adding dropout layers, and applying regularizations to limit the complexity of weights.

4. Different combinations of options on dropout and regularization are attempted, and the best one works at a 0.6 dropout rate, and applying both l1 and l2 regularization on the dense layer with penalties set at 0.05 for both. The resulting network performs as below. Now the overfitting has been much reduced.

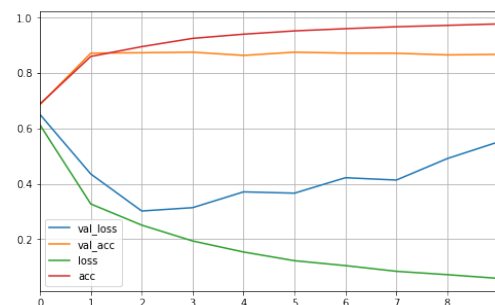


5. When changing the filters from 64 to 250, it's a surprise that the model does not learn. Both loss and accuracy remain at a constant.



Too many filters probably dilute the information contained in the training data, making the dense layer extremely harder to model the text inputs with output.

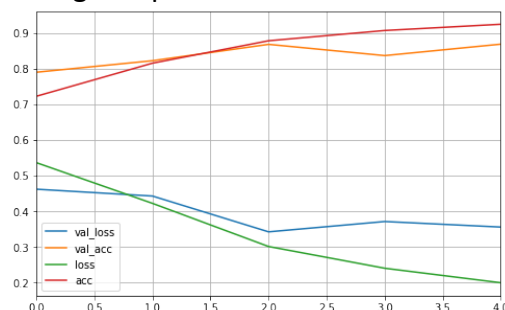
6. Another layer of convolution with BatchNorm is added.



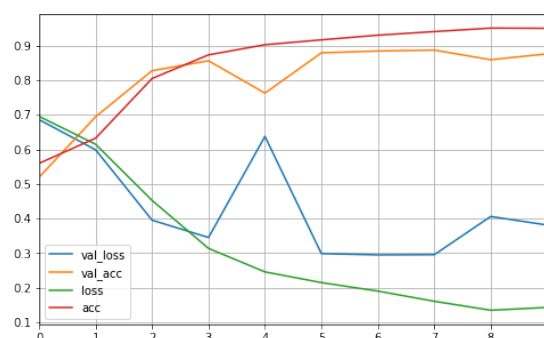
Compared with the baseline, this model has been very similar, except that validation loss grows larger after 6 epochs, indicating more overfitting.

While compared with 2D CNN in lab5, there are different changes. Adding filters would help increase accuracy for 2D CNN as elaborated in lab5, but not here. And adding more layers does not seem to help much in both 1D and 2D CNN. They both see a little bit more overfitting with increased validation loss after 6-7 epochs.

7. The LSTM model is introduced with 2 LSTM layers. Compared to the baseline, it improves the validation accuracy and at the same time diminishing validation loss a lot, despite a little bit overfitting shown in the below training history. Thanks to the structure of LSTM cells that enables prioritizing long sequence information and memorizing parts of them using forget gate, we achieve more accurate information retrieval from the text, and at the same time alleviate the extent of overfitting compared to the baseline.

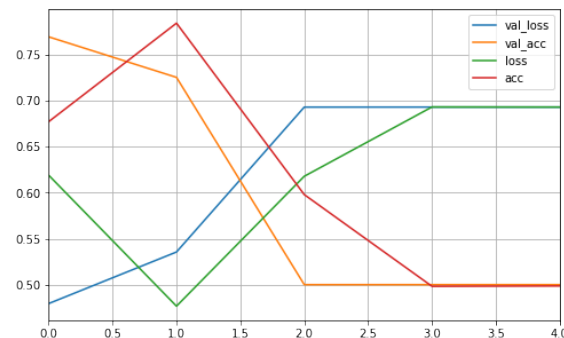


8. Adding one more layer of LSTM generates a rather different performance curve. 10 epochs are trained for thorough analysis. Looking from the training history below, one more LSTM layer brings in lots of weights to learn, so it takes more iterations for gradients to descend.



After 4 epochs, the network shows more overfitting than the 2-layer LSTM. To my understanding, stacking LSTM layers together would increase probability of overfitting, since feeding the output of former LSTM layers into the next layer results in more complexity of the network, while forget gates do not necessarily extract high level information(like convolutional layers do) for the final dense layer modeling.

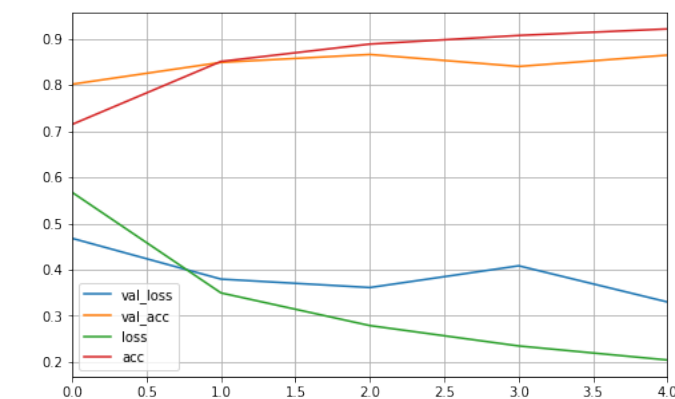
9. Larger LSTM layers are attempted from 128 to 256.



Both training and validation accuracy begin to drop after 2 epochs, and finally converge at .5. While training and validation loss begin to rise after 2 epochs and converge at .7. This is quite similar to the effect of increasing filters for 1D CNN.

It's important that the "unit" parameter we tune here represents the dimensionality of the hidden state that one LSTM cell outputs. Larger hidden units make the cell remember longer sequence and generate more features, thus making the network more complicated. Not sure why the 256 unit LSTM suddenly stops to learn and converges to a random-guess accuracy, probably due to the fact that 256 units are way too much to include for extracting sentiment propensity.

10. Below is the performance of the bi-directional LSTM.



	validation accuracy
baseline	.8806 (10 epochs)
2-layer LSTM	.8697(5 epochs)
Bi-directional LSTM	.8664(5 epochs)

Compared with 2-layer LSTM, the bi-directional LSTM generates very similar validation accuracy trend. While it further alleviates overfitting by achieving smaller validation loss than the 2-layer LSTM.

Bi-directional LSTM works by feeding future info as inputs alongside with past sequence, thus providing the network more context to predict the current output with better accuracy. For sentimental analysis, this could be a very powerful add-on that tries to make predictions based on a more “global” manner, especially when there are attitude transition in the texts that would make sentimental analysis rather complicated.

11. The attention mechanism tries to make the output prediction based on the entire input sequence, with the weights parameterized by alignment scores depicting the correlation between every input and output element. The alignment scores are trained together with the entire RNN by a hidden layer.

The purpose of attention is to further enhance “global” consideration for every single output, since classic sequence models are not good at remembering long sequences and making use of information located at long past. And of course, making use of the entire sequence costs training more weights for alignment scores.

It depends whether this is an optimal approach for analyzing sequence input data. For scenarios where past information plays a vital role, or current output heavily depends on previously mentioned texts, it would be worth the efforts training additional attention layers, such as some context analyzing robots that collect info for all the available history and make instant replies. Common applications such as translation or text prediction could use attention as an approach for performance boost, and it has become an issue of performance-efficiency tradeoff.